HW3P2

Automatic Speech Recognition with an Encoder-Decoder Transformer

11-785: Introduction to Deep Learning (Fall 2026)

Schedule

Release

October 9 2026

- This homework is out of 100 points and will consist of a checkpoint + final submission (both are required to achieve full score on this assignment).
- Join the Kaggle competition (here)

Checkpoint Submission

October 30 2026, 11:59 PM ET

- Complete the **Homework 3 Quiz** on Canvas, achieve **FULL** score on MCQs.
- **ALL** above requirements must be met to achieve full points. If this submission is missed, there will be a 3% deduction.

On-Time Submission

November 6 2026, 11:59 PM ET

- Cutoff scores will be released on Piazza after the checkpoint deadline.
- Submit final scores to **Kaggle by November 6 2026, 11:59 PM ET**.
- Clean up your code notebook, create the final submission zip package, and submit the **output zip file to Gradescope (link) by November 8 2026, 11:59 PM ET**.

Slack Submission

November 13 2026, 11:59 PM ET

- Join and submit your final predictions to the **SLACK** Kaggle Competition.
  (access opens only after the on-time deadline)
- Submit to Gradescope for slack submission (link) with the same submission instructions.
- You are in charge of keeping track of the slack days used; you have a maximum of 10 days.

Start Here

- **Collaboration policy:**
  - You are expected to comply with the University Policy on Academic Integrity and Plagiarism.
  - You are allowed to help your friends debug
  - You are allowed to look at your friends code
  - You are not allowed to type code for your friend
  - You are not allowed to look at your friends code while typing your solution
  - You are not allowed to copy and paste solutions off the internet
  - You are not allowed to import pre-built or pre-trained models
  - You can share ideas but not code, you must submit your own code. All submitted code will be compared to all submitted code in this semester and in previous semesters using MOSS.

  We encourage you to meet regularly with your study group to discuss and work on the homework. You will not only learn more, you will also be more efficient that way. However, as noted above, the actual code used to obtain the final submission must be entirely your own.

- **Overview:**
  - **HW3P2**: This section of the homework is intended on developing skills in implementing an Encoder-Decoder Transformer from scratch for end-to-end Automatic Speech Recognition (ASR). Given sequences of 80-dimensional filterbank features extracted from speech, your model will generate the corresponding text transcriptions. You will incrementally build the components of the hw3lib library (dataset, masks, positional encodings, RMSNorm, attention, sublayers, encoder/decoder layers and the full transformer), verify them with the provided local test suite, and then train your model and decode it using greedy (and optionally beam search) decoding. We will evaluate the model with Character Error Rate (CER) through Kaggle. The competition link can be found below.
  - **HW3P2 Quiz**: You need to take an additional quiz with multiple choices, specific for HW3P2. This quiz can be found on Canvas under Quizzes. It is mandatory to complete this quiz, with a FULL score on the MCQs, before the checkpoint deadline for HW3P2.

- **Submission:**
  - **Canvas Quiz**: Quiz Link
  - **Kaggle**: Competition Link
  - **Gradescope**: Code Submission Link

- **Important Notes:**
  - **[Parameter limit]:** The maximum number of parameters allowed for HW3P2 is 30 million.
  - **[Model constraints]:** You must implement your own Encoder-Decoder Transformer from scratch by completing the components in hw3lib, following the specifications and docstrings in the source files.
  - **[Data constraints]:** You may ONLY use the data provided as part of this homework; using external data is strictly prohibited. The validation set must NOT be used for training.

Homework Objectives

- **If you complete this homework successfully, you would ideally have learned**
  - Understand the fundamentals of Automatic Speech Recognition (ASR), including feature extraction and sequence-to-sequence modeling.
  - Implement positional encoding to incorporate sequence order in transformer models for speech processing.
  - Develop and apply padding masks and attention masks to handle variable-length speech inputs and enforce causal constraints in decoding.
  - Understand the role of multihead self-attention in the encoder to capture temporal dependencies in speech representations and cross-attention in the decoder to attend to encoded features for transcription generation.
  - Implement key transformer components such as self-attention layers, cross-attention layers, feedforward layers, and full encoder and decoder layers.
  - Train a transformer-based ASR model using an encoder-decoder architecture with cross-entropy with an auxiliary CTC loss.
  - Understand progressive training strategies.
  - Decode speech inputs using greedy and beam strategies to generate transcriptions and evaluate model performance using ASR metrics such as CER (Character Error Rate).
  - Tune parameters and hyperparameters to find an optimal solution to the ASR problem.

IMPORTANT NOTE

This write-up focuses on modeling written language. Since written language can be represented at different levels—characters. subwords or words; we will use the term **tokens** to refer to these units. What a token may correspond to will depend on the modeling choice.

The write-up may also include several pseudocode listings. These are intended to convey the logic behind various approaches, but are *not* written in Python and may not be directly translatable. They serve as a conceptual guide; you will need to determine how to implement the logic in Python.

Finally, this document is detailed and covers foundational concepts beyond the specific homework tasks. It includes explanations and pseudocode that may not be directly related to the assignment but are designed to enhance your understanding. The actual homework problems are explicitly listed in certain sections. Please read the document carefully to identify these sections and focus on the relevant material.

Contents

1 Setup & Workflow **6**
- 1.1 About the Handout . . . 6
  - 1.1.1 Dataset Structure . . . 6
  - 1.1.2 Main Library (`hw3lib/`) . . . 6
  - 1.1.3 Reusing Your HW3P1 Attention (`hw3lib/model/attention.py`) . . . 7
  - 1.1.4 Test Suite (`tests/`) . . . 7
- 1.2 Uploading Your Handout . . . 7
  - 1.2.1 Recommended: Using a Private GitHub Repository . . . 8
  - 1.2.2 Using Google Drive . . . 8
  - 1.2.3 Manual Upload . . . 8
  - 1.2.4 Example: Our Preferred Approach . . . 8
- 1.3 Setup Instructions . . . 8

2 Introduction **9**
- 2.1 Training Objective . . . 9
  - 2.1.1 Training with Supervision . . . 9
  - 2.1.2 Preventing Information Leakage . . . 9
- 2.2 Training Process . . . 10
  - 2.2.1 Feature Extraction . . . 10
  - 2.2.2 Transformer Processing . . . 10
  - 2.2.3 Output Logits . . . 10
  - 2.2.4 Softmax and Cross-Entropy Loss . . . 10
- 2.3 Evaluation: Measuring ASR Model Performance . . . 11
  - 2.3.1 Character Error Rate (CER) . . . 11
- 2.4 Decoding: Sequence Generation in ASR . . . 11
  - 2.4.1 Greedy Decoding . . . 11
  - 2.4.2 Beam Search Decoding . . . 11
  - 2.4.3 Language Model Rescoring . . . 11

3 Tasks **12**
- 3.1 Task 1: Automatic Speech Recognition with a Transformer Encoder-Decoder . . . 12

4 Task 1: Automatic Speech Recognition with a Transformer Encoder-Decoder **13**
- 4.1 Data-Processing Components . . . 13
  - 4.1.1 About the `H3Tokenizer` (`hw3lib/data/tokenizer.py`) . . . 13
  - 4.1.2 Dataset Implementation (`hw3lib/data/asr_dataset.py`) . . . 14
- 4.2 Components . . . 15
  - 4.2.1 Positional Encoding (`hw3lib/model/positional_encoding.py`) . . . 15
  - 4.2.2 Rotary Positional Encoding (`hw3lib/model/positional_encoding.py`) . . . 15
  - 4.2.3 Masks (`hw3lib/model/masks.py`) . . . 16
- 4.3 Model Implementations . . . 17
  - 4.3.1 RMSNorm (`hw3lib/model/norms.py`) . . . 17
  - 4.3.2 Transformer Sublayers (`hw3lib/model/sublayers.py`) . . . 17
  - 4.3.3 Transformer Cross-Attention Decoder Layer (`hw3lib/model/decoder_layers.py`) . . . 19
  - 4.3.4 Transformer Self-Attention Encoder Layer (`hw3lib/model/encoder_layers.py`) . . . 19
  - 4.3.5 Encoder-Decoder Transformer (`hw3lib/model/transformers.py`) . . . 21
- 4.4 **Optional/Recommended:** Decoding Implementation . . . 23
- 4.5 Training, Evaluation & Generation . . . 24
  - 4.5.1 ASRTrainer . . . 24

5 Submission **25**

6 Tips & Tricks **26**
- 6.1 Managing Trade-offs through Feature Representation . . . 26
  - 6.1.1 Impact of Tokenization . . . 26

6.1.2 Speech Feature Representation . . . 26
6.1.3 Sequence Downsampling for Efficient Training . . . 26
6.2 Training Strategy 3: Progressive Trainer . . . 26
6.3 Gradient Accumulation . . . 27
6.4 Debugging NaN Loss . . . 28
6.5 Out Of Memory (OOM) . . . 28
6.6 Monitoring Training . . . 28
6.6.1 Visualizing & Interpreting Attention Plots . . . 28
6.6.2 Generated Text Comparison . . . 30
6.7 Miscellaneous . . . 30

1 Setup & Workflow

1.1 About the Handout

In this assignment, we are introducing a new format for assignment delivery, designed to enhance your development workflow. The key motivations for this change are the following.

- **Test Suite Integration**: Your code will be tested in a manner similar to `HWP1's`.
- **Local Development**: You will be able to perform most of your initial development locally, reducing the need for compute resources.
- **Hands-on Experience**: This assignment provides an opportunity to build an end-to-end deep learning pipeline from scratch. We will be substantially reducing the number of abstractions compared to previous assignments.

Notebook Execution Requirements For our provided notebooks to work, your notebook's current working directory must be the same as the handout directory. This is important because the relative imports in the notebooks depend on the current working directory. This can be achieved by:

1. Physically moving the notebook into the handout directory.
2. Changing the notebook's current working directory to the handout directory using the `os.chdir()` function.

Your current working directory should contain the following files:

```
handout_directory/
├── hw3_data_subset/
├── hw3lib/
├── tests/
├── README.md
└── requirements.txt
```

1.1.1 Dataset Structure

We have provided a subset of the dataset for you to use. This subset allows you to implement and test your code locally while maintaining the same structure as the original dataset. The dataset is organized as follows:

```
hw3_data_subset/
└── hw3p2_data/ ..... For end-to-end speech recognition
    ├── dev-clean/
    │   ├── fbank/
    │   └── text/
    ├── test-clean/
    │   └── fbank/
    └── train-clean-100/
        ├── fbank/
        └── text/
```

1.1.2 Main Library (`hw3lib/`)

For `HW3P1`, you implemented Softmax, scaled dot product attention and multi-head attention. In `HW3P2`, you will expand on what you have built by implementing and training an encoder-decoder transformer model for speech recognition.

You will implement components in a way that they can be reusable. You should see the following files in the `hw3lib/` directory (`__init__.py` files are omitted):

```
hw3lib/
└── data/
    ├── asr_dataset.py
    ├── tokenizer.py
    ├── tokenizer_jsons/
    └── verify_dataloader.py
```

```
├── decoding/
│   └── sequence_generator.py
├── model/
│   ├── attention.py
│   ├── decoder_layers.py
│   ├── encoder_layers.py
│   ├── masks.py
│   ├── norms.py
│   ├── positional_encoding.py
│   ├── speech_embedding.py
│   ├── sublayers.py
│   └── transformers.py
├── trainers/
│   ├── asr_trainer.py
│   └── base_trainer.py
└── utils/
    ├── create_lr_scheduler.py
    └── create_optimizer.py
```

1.1.3 Reusing Your HW3P1 Attention (`hw3lib/model/attention.py`)

Unlike previous homeworks, `HW3P2` does **not** include a separate `mytorch/` library. Instead, you will port the NumPy `mytorch.nn.MultiHeadAttention` you implemented in `HW3P1` to PyTorch, in `hw3lib/model/attention.py`. The method names and the `forward` signature are unchanged, so we recommend keeping your `HW3P1` implementation open beside the new file.

- **Carried over:** the `forward` signature, the `_split_heads`, `_concat_heads` and `_merge_masks` methods, and the mask convention (`True` marks a position to ignore).
- **Changes:** the class subclasses `nn.Module`, the `mytorch` `Linear` layers become `nn.Linear`, NumPy operations become their `torch` equivalents, and `forward` now also returns the attention weights.
- **New:** optional hooks for rotary positional embeddings and QK-Norm, both applied to the queries and keys only, after `_split_heads`.

1.1.4 Test Suite (`tests/`)

In `HW3P2`, you will be provided with a test suite to verify your implementation. The `tests/` directory should contain the following files:

```
tests/
├── test_attention.py
├── test_dataset*.py
├── test_decoding.py
├── test_decoderlayer*.py
├── test_encoderlayer*.py
├── test_mask*.py
├── test_norms.py
├── test_positional_encoding.py
├── test_rope.py
├── test_sublayer*.py
├── test_transformer*.py
└── testing_framework.py
```

1.2 Uploading Your Handout

Regardless of the environment you work in, you must have a reliable way of uploading your handout to your working directory for the project to function correctly. Below are some recommended methods:

1.2.1 Recommended: Using a Private GitHub Repository

The most reliable way to manage your handout is by storing it in a **private** GitHub repository. Follow these steps:

1. Set up a Personal Access Token (PAT) for your repository.
2. Clone your repository using the following command:

```
!git clone https://<your_personal_access_token>@github.com/<username>/<repository>.git
```

1.2.2 Using Google Drive

Another approach is to store your handout in Google Drive and retrieve it in Colab using one of the following methods:

- **Mount Google Drive**: If using Colab, Use the built-in Colab feature to mount your drive and manually move your files to the working directory.
- **Use gdown**: If your file is publicly accessible with edit permissions, you can download it using gdown.

1.2.3 Manual Upload

For a quick solution, you can manually upload your handout into the filesystem.

WARNING: YOU MUST ENSURE CONSISTENCY. You will be implementing components of `hw3lib` inside the project directory throughout `HW3P2`. It is **your responsibility** to ensure that changes remain consistent across both parts.

1.2.4 Example: Our Preferred Approach

Below is a sample script demonstrating a preferred way to clone and manage your repository securely:

```
import os

# Store your Personal Access Token securely in an environment variable
os.environ['GITHUB_TOKEN'] = "your_personal_access_token"

GITHUB_USERNAME = "your-username"
REPO_NAME      = "IDL-HW3"
TOKEN = os.environ.get("GITHUB_TOKEN")
repo_url       = f"https://{TOKEN}@github.com/{GITHUB_USERNAME}/{REPO_NAME}.git"

# Clone the repository
!git clone {repo_url}

# To pull your latest changes (Ensure you are in the correct directory)
!git pull
```

1.3 Setup Instructions

Setup instructions for **Local**, **PSC**, **Colab**, and **Kaggle** environments are available in the **README.md** file within the handout, as well as in the **HW3P2_Student_Starter_Notebook_F26.ipynb** notebooks.

2 Introduction

Welcome to HW3P2! In this assignment, you will implement components to build an **encoder-decoder transformer** model from scratch, train it on a speech-to-text task, and use it for automatic speech recognition. This will be a monumental task, but we promise it will be a rewarding one. We will break down this task into several smaller tasks, each of which will build on the previous. We will begin with an overview of the training objective, evaluation, and inference before breaking down the implementation into several smaller tasks.

WARNING: This assignment is designed to be significantly more challenging than previous assignments. We recommend starting early and reaching out to Piazza if you get stuck.

2.1 Training Objective

We aim to train an **encoder-decoder transformer** to model the probability of a sequence of text tokens given an input sequence of acoustic features:

$$P(y_1, ..., y_M | x_1, ..., x_N) = \prod_{t=1}^{M} P(y_t | y_1, ..., y_{t-1}, x_1, ..., x_N) \tag{1}$$

where $x_1, ..., x_N$ represents a sequence of acoustic feature frames extracted from speech, and $y_1, ..., y_M$ represents the corresponding text tokens. The conditional probability $P(y_t | y_1, ..., y_{t-1}, x_1, ..., x_N)$ is modeled using our transformer.

2.1.1 Training with Supervision

We train the model using a large corpus of **paired speech and text**. Given an input speech utterance represented as a sequence of feature frames $x_1, ..., x_N$, the target is a sequence of tokens $y_1, ..., y_M$ representing the transcription.

Example For an audio clip containing the spoken phrase:

> "I swam across the river"

the target text sequence would be:

- **Input:** Speech features like filterbanks (HW3P2) or MFCCs (HW1P2) extracted from the speech signal.
- **Target:** ["I", "swam", "across", "the", "river"]

We can use a decoder model to generate the token sequence corresponding to the speech transcript, token by token. This decoder needs to be conditioned on the entire input sequence corresponding to the speech features! An encoder transformer can be used to process and map the entire input sequence into a suitable internal representation before decoding starts.

2.1.2 Preventing Information Leakage

To ensure that the decoder **does not access future tokens during training**, we apply the following techniques:

1. **Start and End Tokens:**
   - Prepend a special **start-of-sequence** token (⟨SOS⟩) to the **target sequence** to signal the beginning of decoding.
   - Append an **end-of-sequence** token (⟨EOS⟩) to mark the end of the sequence and allow the model to learn when to stop decoding.
2. **Padding and Pad Masks:**
   - Since GPUs require **fixed-length tensors**, shorter sequences are **padded** using a special padding token (⟨PAD⟩).
   - A **pad mask** ensures that these padded positions do not contribute to model predictions, preventing unwanted bias.
3. **Causal Masking in the Decoder:**

- The decoder uses a **causal mask** (also called a look-ahead mask) to prevent each position from attending to future tokens, ensuring autoregressive generation.
- This masking enforces left-to-right decoding, where each token is predicted based only on previous tokens in the sequence.

Handling Variable-Length Inputs in the Encoder: Unlike the decoder, the encoder processes the entire input sequence without causal restrictions. However, padding is still necessary to fit variable-length sequences into a batch, requiring a **pad mask** to prevent padded positions from affecting the encoder's learned representations.

2.2 Training Process

2.2.1 Feature Extraction

Each speech signal is converted into a sequence of F-dimensional feature vectors using a feature extraction pipeline:

$$x_1, ..., x_N \in \mathbb{R}^F \tag{2}$$

where:

- F is the dimensionality of the extracted features (e.g., Mel spectrogram, MFCCs).
- Each frame x_i is a feature vector representing a short segment of the speech signal.

We have already taken care of this for you! In this context, you will be working with sequences of 80-dimensional filterbank features as your speech representations.

2.2.2 Transformer Processing

The input sequence of **F-dimensional features** is optionally processed using shallow convolutional layers to capture local dependencies and recurrent layers to model temporal patterns to produce a sequence of **D-dimensional features**. Position information is then added, either as sinusoidal encodings added to this sequence, or as rotary encodings (RoPE, the default) applied inside each self-attention layer (Section 4.2.2). The features then pass through a stack of **encoder layers**, where **self-attention** and **feedforward transformations** are applied.

The encoder outputs a sequence of hidden states, which serve as input to the **decoder**. The decoder employs **masked self-attention** to process its own outputs autoregressively and **cross-attention** to attend to the encoder's representations, refining its predictions at each step.

2.2.3 Output Logits

The model produces a sequence of hidden states $\tilde{y}_1, ..., \tilde{y}_M$, each of dimensionality D. To obtain logits over the vocabulary (K), we apply a **final linear transformation**:

$$Y = XW^{(p)} \tag{3}$$

where:

- $W^{(p)} \in \mathbb{R}^{D \times K}$ is the **projection matrix**.
- $X \in \mathbb{R}^{M \times D}$ is the output of the hidden state of the decoder.
- $Y \in \mathbb{R}^{M \times K}$ are **logits**, representing unnormalized scores over the vocabulary.

2.2.4 Softmax and Cross-Entropy Loss

We can apply the **softmax function** to obtain probabilities:

$$P(y_t | y_1, ..., y_{t-1}, x_1, ..., x_N) = \text{softmax}(Y_t) \tag{4}$$

However, **we do not explicitly compute softmax in the code**. Instead, PyTorch's `nn.CrossEntropyLoss`:

- Takes the **raw logits** Y and the **ground-truth token sequence**.

- **Internally applies softmax** before computing the loss.

This training approach **maximizes the likelihood of the training data** and enables the model to learn meaningful speech-to-text mappings.

2.3 Evaluation: Measuring ASR Model Performance

To evaluate the quality of our **encoder-decoder transformer**, we need a metric that captures how well the model predicts the transcription given the input speech features.

2.3.1 Character Error Rate (CER)

Error rates measure the difference between the model's predicted transcript and the ground truth. In this homework, we use the **Character Error Rate (CER)**:

$$CER = \frac{S + D + I}{N} \tag{5}$$

where:

- S is the number of character substitutions.
- D is the number of character deletions.
- I is the number of character insertions.
- N is the total number of characters in the reference transcript.

A lower CER indicates a better ASR model. Your goal is to train your transformer to minimize this error.

2.4 Decoding: Sequence Generation in ASR

After training the model, we can use it to transcribe speech into text. During inference, the transformer decoder outputs a probability distribution over possible tokens at each time step. To construct the final transcription, a decoding strategy must be employed to select tokens based on these probabilities. Below are common decoding strategies used in ASR:

2.4.1 Greedy Decoding

Greedy decoding selects the most probable token at each time step without considering future probabilities. While efficient with a complexity of $O(KN)$, it is prone to errors since it does not optimize the overall sequence probability. This often leads to suboptimal transcriptions, especially in ambiguous contexts.

2.4.2 Beam Search Decoding

Beam search maintains B candidate hypotheses at each step, selecting the top B most probable sequences based on cumulative probability. This approach balances efficiency and accuracy by pruning lower-probability sequences while exploring multiple possibilities. With a complexity of $O(BKN)$, it provides better transcriptions than greedy decoding but is more computationally demanding.

2.4.3 Language Model Rescoring

To further improve transcription accuracy, an external language model (LM) can be used to rescore beam search candidates. The LM assigns additional probabilities based on linguistic context, helping resolve ambiguities and improving fluency. This is particularly beneficial for ASR in noisy conditions or domain-specific applications.

3 Tasks

Your model performance in HW3P2 will be graded on Kaggle. Download the starter code. The following sections describe the tasks you will need to complete for this assignment. Let us begin with an overview.

3.1 Task 1: Automatic Speech Recognition with a Transformer Encoder-Decoder

- Familiarize yourself with the `tokenize`, `encode` and `decode` methods of `H3Tokenizer` class.
- Implement the `ASRDataset` class to load and pre-process the data.
- Implement the key components: `PositionalEncoding`, `RotaryPositionalEncoding`, `CausalMask` and `PadMask`.
- Implement `RMSNorm`.
- Port your `HW3P1` `MultiHeadAttention` to PyTorch.
- Implement the Transformer Sublayers: `SelfAttentionLayer`, `CrossAttentionLayer`, `SwiGLU` and `FeedForwardLayer`.
- Implement Transformer layers: `CrossAttentionDecoderLayer` and `SelfAttentionEncoderLayer`.
- Implement the `EncoderDecoderTransformer` class.
- Implement `Greedy` decoding.
- **Optional/Recommended:** Implement `Beam` decoding.
- Implement parts of `ASRTrainer`.
- Train the model on the dataset.
- Transcribe speech features in the test set and submit to Kaggle.

Restrictions:

- You may **only** use the data provided as part of this homework; using external data is strictly prohibited.
- The validation set **must not** be used for training.
- You are required to use at least **greedy decoding** for speech transcription. **Beam search** is optional but recommended.
- You may **only** use the standard **pre-norm Transformer** architecture; other transformer variants (eg. Conformer, E-Branchformer, etc.) are not permitted.
- The total parameter count of your model must not exceed **30M**.

NOTE: All implementations have detailed specifications, implementation details, and hints in their respective source files. Make sure to read all the comments and docstrings in their entirety to understand the implementation details!

4 Task 1: Automatic Speech Recognition with a Transformer Encoder-Decoder

In **HW3P2**, you will incrementally implement components of **hw3lib** to build and train an Encoder-Decoder Transformer for end-to-end speech recognition.

The following sections will walk you through the steps needed to complete the latter.

REMINDER: All implementations have detailed specifications, implementation details, and hints in their respective source files. Make sure to read all the comments and docstrings in their entirety to understand the implementation details!

4.1 Data-Processing Components

To begin, we need essential tools for converting raw speech-text pairs into a format suitable for training an ASR model. A typical preprocessing pipeline performs the following steps:

1. Load paired speech features (e.g., 80-dimensional filterbanks) and text data into memory.
2. Normalize the extracted features for consistency across different recordings.
3. Tokenize the text transcripts (e.g., at the character or subword level).
4. Construct a vocabulary dictionary mapping each token to a numerical index.
5. Convert text transcripts into sequences of numerical indices.
6. Align speech features and text sequences, ensuring compatibility for model training.

4.1.1 About the `H3Tokenizer` (`hw3lib/data/tokenizer.py`)

Tokens are the smallest units of text your model will process. Each time step corresponds to one token, but what constitutes a "token" depends on the tokenization strategy you choose. For example, the sentence "Baby needs a new pair of shoes" can be represented as:

- A sequence of 7 word-level tokens, drawn from a large vocabulary (typically tens or hundreds of thousands of words).
- Or as a sequence of 30 character-level tokens, using a much smaller vocabulary (e.g., 256 ASCII characters).

For this assignment, you will convert each text transcript into:

- A sequence of tokens.
- A corresponding sequence of numerical indices, where each index represents the token's position in the vocabulary.

These numerical sequences will serve as inputs to and outputs from your model. During inference or generation, you will also need to reverse this process—converting indices back to tokens, and then reconstructing the original text.

We have provided the `H3Tokenizer` class in `hw3lib/data/tokenizer.py` to handle tokenization for `HW3P2`.

You will be working with two broad categories of tokenization strategies:

- **Character-level tokenization:** Each character in the language is treated as a token, resulting in a small vocabulary but longer token sequences for each sentence.
- **Subword tokenization:** Splits words into smaller reusable subword units. The subword method uses Byte Pair Encoding (BPE) to learn and apply subword merges. This approach allows for a compact vocabulary that still captures uncommon or rare words by representing them as a combination of subwords.

Selecting an appropriate tokenization strategy is crucial, as it affects the model's vocabulary size, memory efficiency, and handling of rare or out-of-vocabulary words. As part of this assignment, you will explore how different tokenization strategies affect model performance in `HW3P2`.

`H3Tokenizer` supports the following tokenization strategies:

- Character-level tokenization
- Subword tokenization with a vocabulary size of 1,000
- Subword tokenization with a vocabulary size of 5,000
- Subword tokenization with a vocabulary size of 10,000

Before proceeding, familiarize yourself with the `H3Tokenizer` class and its key methods:

- `tokenize`
- `encode`
- `decode`

Their documentation can be found in their source files. You will use these methods both when preparing datasets and during model decoding.

4.1.2 Dataset Implementation (`hw3lib/data/asr_dataset.py`)

For HW3P2, you will be working with the `hw3p2_data` subdirectory of the downloaded `hw3p2-data` directory (e.g., `/content/hw3p2-data/hw3p2_data` on Colab, or `/local/hw3p2-data/hw3p2_data` on PSC). Set this path as `root` in the notebook config. The structure is organized as follows:

```
hw3p2-data/.
└── hw3p2_data/
    ├── train-clean-100/
    │   ├── fbank/
    │   └── text/
    ├── dev-clean/
    │   ├── fbank/
    │   └── text/
    └── test-clean/
        └── fbank/
```

The `train-clean-100`, `dev-clean`, and `test-clean` directories contain the dataset splits. Each split consists of:

- Speech features stored in `.npy` format within the `fbank` subdirectory.
- Text transcripts stored in `.npy` format within the `text` subdirectory.

Note that the `test-clean` directory contains only speech features. Your model will generate transcriptions for these samples, which will be evaluated on Kaggle.

To work with this dataset, you will use the `ASRDataset` class provided in `hw3lib/data/asr_dataset.py`. This class is designed to:

- Load the `.npy` feature files containing filterbank representations.
- Load the corresponding transcript files (except for the test partition).
- Tokenize the transcripts using a provided tokenizer.
- Apply normalization strategies to the features:
  - **Global MVN**: Normalization using global mean and variance computed from training data.
  - **Cepstral**: Per-utterance mean and variance normalization.
  - **None**: No normalization applied.
- Apply **SpecAugment** data augmentation (only for training), including:
  - Time masking (masking random time steps).
  - Frequency masking (masking random frequency bands).
- Prepare two versions of each tokenized transcript:

  - A **shifted** version with a Start-of-Sequence (SOS) token prepended.
  - A **golden** version with an End-of-Sequence (EOS) token appended.
- Track dataset statistics such as total characters, tokens, and sequence lengths.
- Provide a custom `collate_fn` function for batching data correctly by:
  - Padding features and transcripts to batch-uniform lengths.
  - Providing lengths for packed sequence processing and mask generation.
  - Ensuring proper tensor types.

Your task is to complete parts of the `__init__` method and fully implement the `__len__`, `__getitem__`, and `collate_fn` methods **according to the provided specifications in the source file**. Your implementation should ensure sequences are properly aligned, padded, and formatted for training and evaluating an automatic speech recognition model.

Run the command in the **Dataset Implementation** section of `HW3P2_Student_Starter_Notebook_F26.ipynb` to test your implementation incrementally.

4.2 Components

4.2.1 Positional Encoding (`hw3lib/model/positional_encoding.py`)

Transformers lack an inherent understanding of token order, so positional encodings are added to the input embeddings. These encodings inform the model about the sequential nature of the data and help differentiate between tokens based on their positions.

In this section, you will implement a fixed positional encoding scheme based on sine and cosine functions, as introduced in the original Transformer paper (*Vaswani et al., 2017*).

Given an input representation consisting of d-dimensional embeddings for T tokens, positional encoding generates a matrix $\mathbf{P} \in \mathbb{R}^{T \times d}$ of the same shape. The elements of $\mathbf{P}$ are computed as follows:

$$\mathbf{P}_{t,2i} = \sin\left(\frac{t}{10000^{\frac{2i}{d}}}\right), \quad \mathbf{P}_{t,2i+1} = \cos\left(\frac{t}{10000^{\frac{2i}{d}}}\right)$$

As shown in the equation above, for each time step t, the positional encoding uses sine on even embedding dimensions and cosine on odd embedding dimensions. Their wavelengths form a geometric progression from 2π to $10000 \cdot 2\pi$. The generated positional vectors are then added to input vectors by vector addition.

Implement the following methods of the `PositionalEncoding` class in `hw3lib/model/positional_encoding.py`:

- `create_pe_table`: Constructs the positional encoding matrix $\mathbf{P}$ of shape (max_len, d_model) where:
  - `max_len`: The maximum length of input sequences.
  - `d_model`: The dimensionality of each token embedding.
- `forward`: Adds the positional encoding matrix $\mathbf{P}$ to the input embeddings before feeding them into the model.

NOTE: Remember that the positional encoding matrix should be registered as a buffer (e.g., using `self.register_buffer()`) to ensure it is not treated as a learnable parameter. The base table may be constructed with the shape (max_len, d_model), but the stored buffer should have the shape $(1, max_len, d_model)$ so it broadcasts correctly across the batch dimension during the forward pass.

4.2.2 Rotary Positional Encoding (`hw3lib/model/positional_encoding.py`)

Rotary Positional Encoding (RoPE) (*Su et al., 2021*) is an alternative to the sinusoidal encoding above. Instead of adding position information to the input embeddings, RoPE rotates the queries and keys inside each attention head by an angle that depends on their position. As a result, the attention score between a query at position m and a key at position n depends only on their **relative position** $n - m$.

For a vector x of dimension d (`head_dim`) at position m, the rotation is:

$$\text{RoPE}(x, m) = x \odot \cos(m\theta) + \texttt{rotate_half}(x) \odot \sin(m\theta), \quad \theta_i = 10000^{-\frac{2i}{d}}$$

where each dimension i is paired with dimension $i + d/2$, and both share the angle $m\theta_i$.

Implement the following methods of the `RotaryPositionalEncoding` class:

- `create_rope_table`: Builds the cos and sin tables of the angles $m\theta_i$ for every position m up to `max_len`.
- `rotate_half`: Swaps the two halves of the last dimension and negates the new first half: $[x_1, x_2] \to [-x_2, x_1]$.
- `forward`: Applies the rotation above to the queries and keys, and returns them.

NOTE: RoPE is applied to the queries and keys **only** (never the values), after they are projected and split into heads. It is used only in the self-attention layers. Register the tables as buffers of shape $(1, 1, max_len, d)$.

4.2.3 Masks (`hw3lib/model/masks.py`)

Before implementing the encoder-decoder transformer, we need to create helper functions for masking, which will be implemented in `hw3lib/model/masks.py`.

NOTE: While it's possible to implement these functions naively using a `for` loop, we recommend using vectorized operations to speed up the implementation, as these functions will be called repeatedly during training. Some useful PyTorch functions for this task include `torch.ones_like`, `torch.arange`, `torch.expand`, `torch.repeat`, `torch.tril`, and various boolean operations (, `>=`, `==`, `!=`, etc.) on tensors. It is possible to implement these functions using PyTorch's built-in functions with just 3-5 lines of code each.

Causal Mask: Implement the `CausalMask` function in `hw3lib/model/masks.py`. This function should take a padded batch of input sequences with shape $(N, T, \dots)$ or (N, T) and return a mask of shape (T, T), where T is the sequence length. The mask should be `True` for positions that should be masked (i.e., positions that should not attend to future tokens), and `False` for positions that are allowed.

For example, if we have a batch of two sequences with variable lengths, padded to the same length (with `0` as the padding token):

$$\text{input} = \begin{bmatrix} 1 & 2 & 3 & 0 & 0 \\ 1 & 2 & 0 & 0 & 0 \end{bmatrix}$$

Then the output mask will be:

$$\text{mask} = \begin{bmatrix} \text{False} & \text{True} & \text{True} & \text{True} & \text{True} \\ \text{False} & \text{False} & \text{True} & \text{True} & \text{True} \\ \text{False} & \text{False} & \text{False} & \text{True} & \text{True} \\ \text{False} & \text{False} & \text{False} & \text{False} & \text{True} \\ \text{False} & \text{False} & \text{False} & \text{False} & \text{False} \end{bmatrix}$$

NOTE: As with the previous example, while this illustrates the (N, T) case, your function should also support the $(N, T, \dots)$ case.

Pad Mask: Implement the `PadMask` function in `hw3lib/model/masks.py`. This function should take a padded batch of input sequences with shape $(N, T, \dots)$ or (N, T) and a tensor of input lengths with shape $(N,)$, and return a mask of shape (N, T) where N is the batch size and T is the sequence length. The mask should be `True` for padding positions and `False` for non-padding positions.

For example, if we have a batch of two sequences of variable lengths, padded to the same length within the batch (with `0` as the padding token):

$$\text{input} = \begin{bmatrix} 1 & 2 & 3 & 0 & 0 \\ 1 & 2 & 0 & 0 & 0 \end{bmatrix}$$

And the lengths tensor:

$$\text{lengths} = \begin{bmatrix} 3 \\ 2 \end{bmatrix}$$

Then the output mask will be:

$$\text{mask} = \begin{bmatrix} \text{False} & \text{False} & \text{False} & \text{True} & \text{True} \\ \text{False} & \text{False} & \text{True} & \text{True} & \text{True} \end{bmatrix}$$

NOTE: While this example illustrates the (N, T) case, your function implementation should also support the $(N, T, \dots)$ case.

4.3 Model Implementations

The following sections will guide you through the incremental process of building a Encoder-Decoder Transformer model. Specifically, we will implement the pre-norm variant of the encoder-decoder transformer architecture. Before we begin, let's define some key terminology that will be used throughout the architecture construction process:

Terminology:

- **Sublayer:** A fundamental building block within a Transformer layer. Examples include self-attention, cross-attention and feedforward layers.
- **Layer:** A complete unit in a Transformer model, consisting of multiple sublayers (e.g., a Cross-Attention Decoder Layer that includes self-attention, cross-attention and feedforward layers).
- **Transformer:** A deep neural network architecture composed of multiple layers along with additional components like embedding layers, positional encoding layers, etc.

NOTE: As you incrementally implement each component, you can test your implementations using the commands provided in the `Model Implementations` section of `HW3P2_Student_Starter_Notebook_F26.ipynb`.

4.3.1 RMSNorm (`hw3lib/model/norms.py`)

Root Mean Square Normalization (RMSNorm) is a simpler version of LayerNorm. LayerNorm subtracts the mean and divides by the standard deviation. RMSNorm skips the mean subtraction and only divides by the root mean square of the vector. In practice this works just as well, and it is cheaper to compute.

For a vector x of dimension d, RMSNorm is computed over the last dimension as:

$$\text{RMSNorm}(x) = \frac{x}{\sqrt{\frac{1}{d}\sum_{i=1}^{d} x_i^2 + \epsilon}} \odot g$$

where g is a learnable per-feature scale, and ϵ is a small constant for numerical stability. Unlike LayerNorm, RMSNorm has **no bias**.

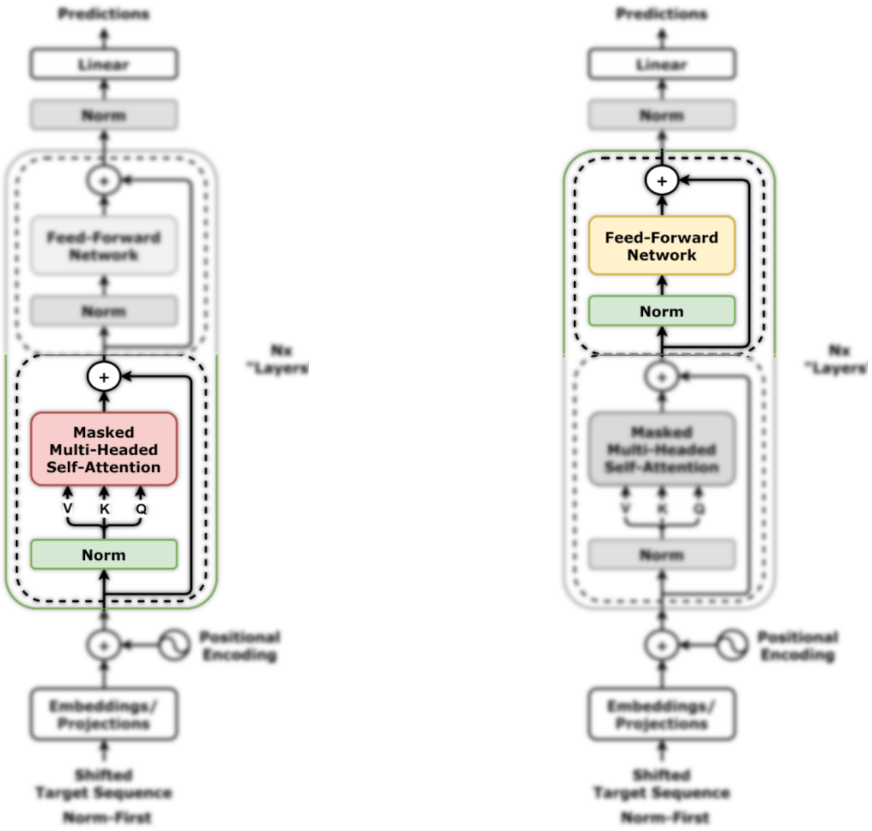
Implement the following methods of the `RMSNorm` class:

- `__init__`: Creates the learnable scale g of shape (d), initialized to ones.
- `forward`: Applies the equation above and returns a tensor of the same shape as the input.

NOTE: Compute the root mean square in **float32**, then cast the result back to the input's dtype. We train with mixed precision, so inputs arrive as float16, where x^2 overflows for $|x| > 256$ and the output silently becomes zero. The same `RMSNorm` module is also reused as **QK-Norm** inside the attention layer. See the docstrings for full details.

4.3.2 Transformer Sublayers (`hw3lib/model/sublayers.py`)

In the following section, you will implement the sublayers required for the pre-norm variant of the encoder-decoder Transformer.



(a) Self-Attention Sublayer Architecture

(b) FeedForward Sublayer Architecture

Figure 1: Transformer Sublayers

Multi-head Attention: In `hw3lib/model/attention.py`, you need to implement the `MultiHeadAttention` class. This works fundamentally the same way as what you did in **HW3P1**, with a few key differences:

- `backward` and `init_weights` are no longer needed, since PyTorch's autograd computes the gradients.
- The `forward` method returns both the output and the attention weights (averaged over the heads).
- Rotary embeddings (Section 4.2.2) and QK-Norm are introduced. When enabled, they are applied to the queries and keys only, after `_split_heads`.

Check the code documentation for details.

SelfAttentionLayer: Implement the `SelfAttentionLayer` class in `hw3lib/model/sublayers.py`. Refer to Figure 1a (just the colored portion) for the layer architecture. This class will contain the logic for the self-attention mechanism, where the queries, keys and values all come from the same input. **Follow the specifications and doc strings in the source file carefully for our tests to work.**

FeedForwardLayer: Implement the `FeedForwardLayer` class in `hw3lib/model/sublayers.py`. Refer to Figure 1b (just the colored portion) for the layer architecture. The feed-forward network is selected with `ffn_type`: `swiglu` (default) uses the `SwiGLU` class described below, and `gelu` uses a standard two-layer network with a GELU activation. **Follow the specifications and doc strings in the source file carefully for our tests to work.**

CrossAttentionLayer: Implement the `CrossAttentionLayer` class in `hw3lib/model/sublayers.py`. Refer to Figure 2 (just the colored portion) for the layer architecture. This class will contain the logic for the cross-attention mechanism, where the queries come from the decoder and the keys and values come from the encoder output. Rotary embeddings are **not** used here, since the queries and keys come from two different sequences (text and speech). **Follow the specifications and doc strings in the source file carefully for our tests to work.**

SwiGLU: SwiGLU is a gated feed-forward network. Instead of one projection followed by an activation (as in the GELU network), it uses one projection to gate another:

$$\mathrm{SwiGLU}(x) = W_{\mathrm{down}}\big(\mathrm{SiLU}(W_{\mathrm{gate}}\, x) \odot W_{\mathrm{up}}\, x\big)$$

where $\mathrm{SiLU}(z) = z \cdot \sigma(z)$ is the Swish activation you implemented in **HW1P1**, and all three linear layers have no bias. Since SwiGLU needs three weight matrices where the GELU network needs two, its hidden dimension is $\lfloor \frac{2}{3} d_{ff} \rfloor$, so both networks have about the same number of parameters. You will implement the `__init__` and `forward` functions in `hw3lib/model/sublayers.py`. Check comments in the starter code for more guidance.

4.3.3 Transformer Cross-Attention Decoder Layer (`hw3lib/model/decoder_layers.py`)

Implement the `CrossAttentionDecoderLayer` class in `hw3lib/model/decoder_layers.py`. Refer to Figure 3a (just the colored portion) for the layer architecture. This class just contains the SelfAttentionLayer, CrossAttentionLayer and FeedForwardLayer as submodules. **Follow the specifications and doc strings in the source file carefully for our tests to work.**

4.3.4 Transformer Self-Attention Encoder Layer (`hw3lib/model/encoder_layers.py`)

Implement the `SelfAttentionEncoderLayer` class in `hw3lib/model/encoder_layers.py`. Refer to Figure 3b (just the colored portion) for the layer architecture. This class just contains the `SelfAttentionLayer` and `FeedForwardLayer` as submodules. The implementation is similar to the `CrossAttentionDecoderLayer` from the previous section, with two key differences:

- There is no cross-attention sublayer, since the encoder only attends to its own input.
- There is no causal mask: every position can attend to every other position in the input sequence. Only the padding mask is needed.

Follow the specifications and doc strings in the source file carefully for our tests to work.

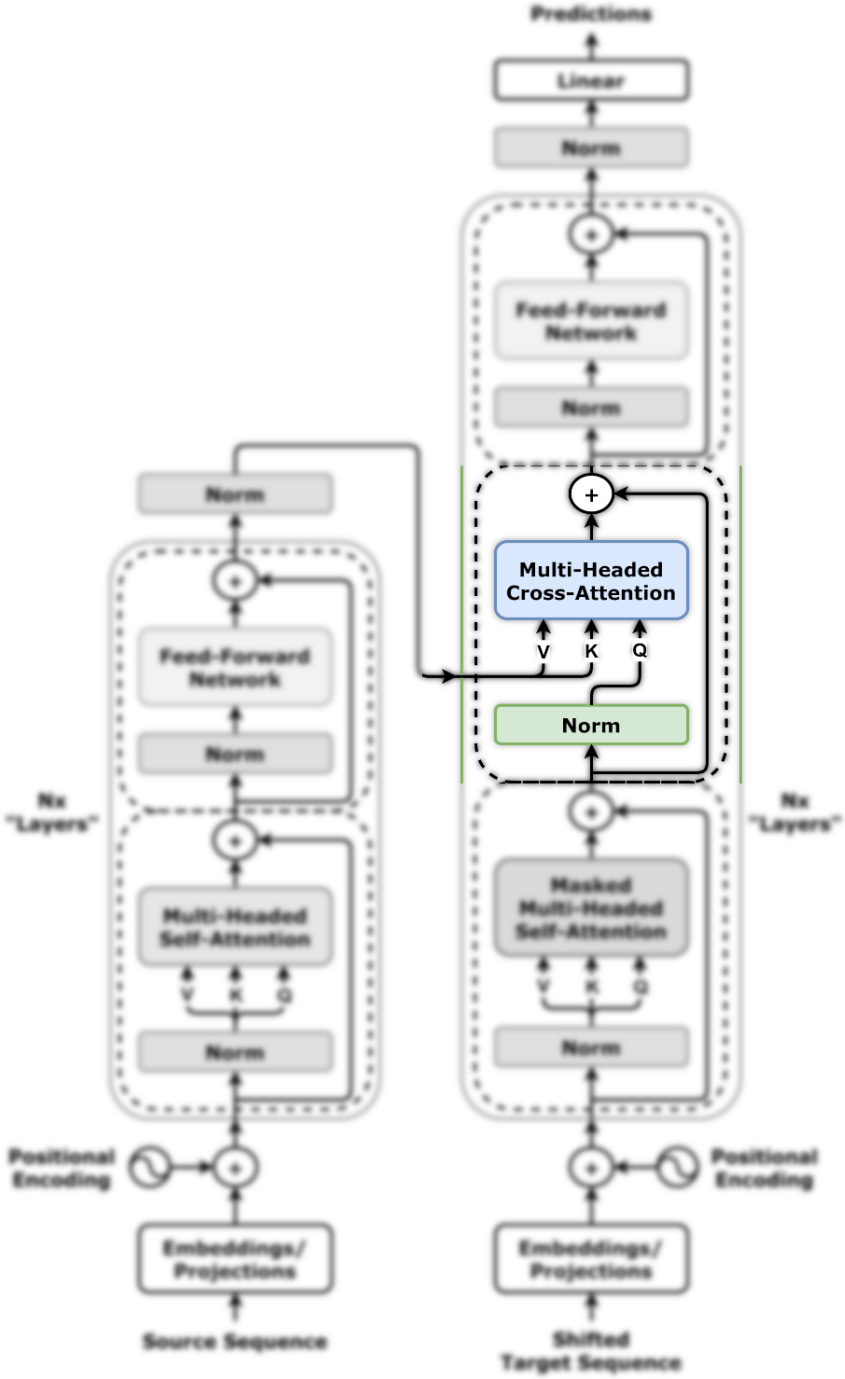
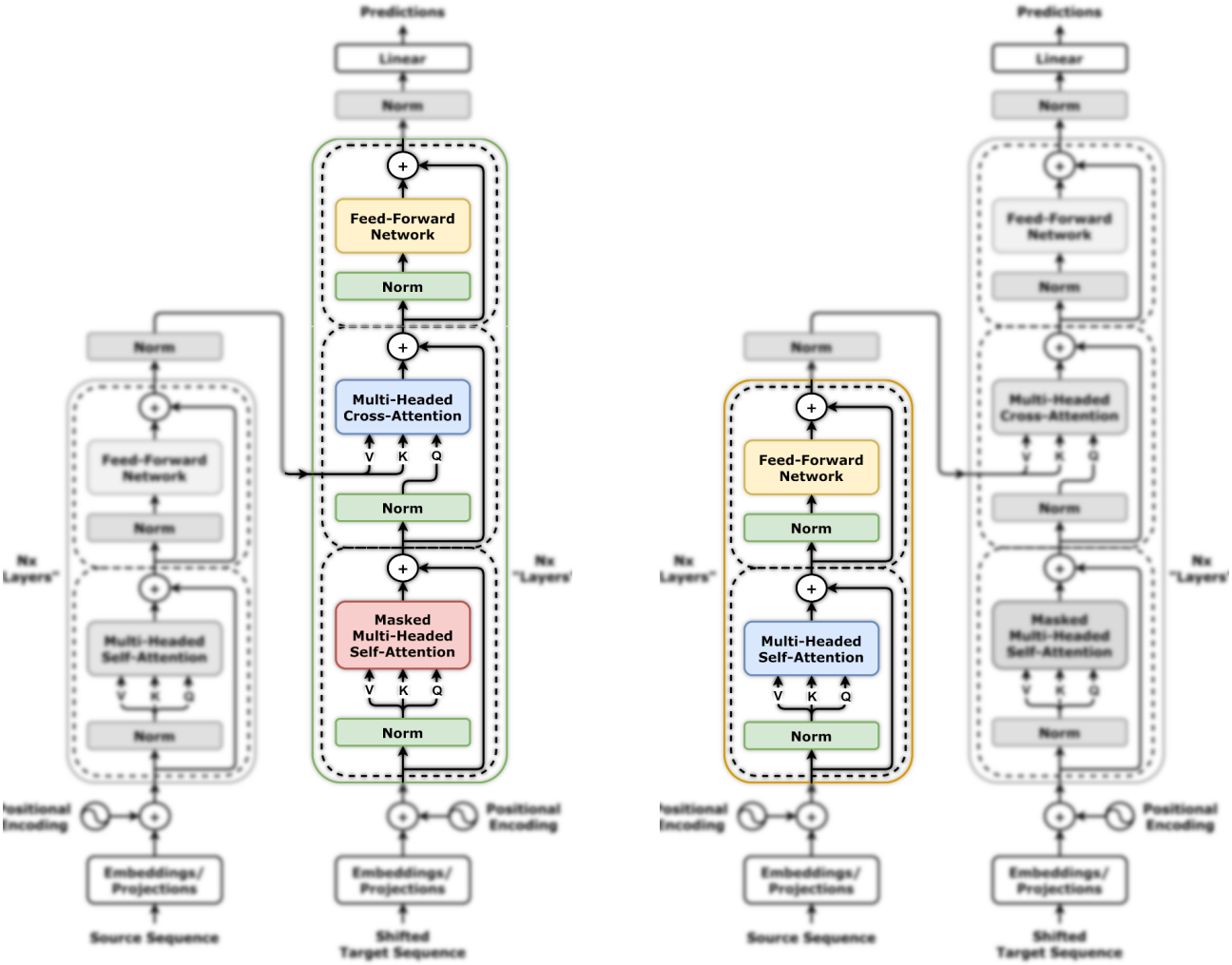


Figure 2: CrossAttention Layer



(a) Cross-Attention Decoder Layer Architecture

(b) Self-Attention Encoder Layer architecture

Figure 3: Transformer Layers

4.3.5 Encoder-Decoder Transformer (`hw3lib/model/transformers.py`)

In this section, we integrate all the components to construct the Encoder-Decoder Transformer. Specifically, implement the `EncoderDecoderTransformer` class in `hw3lib/model/transformers.py`. Refer to Figure 4 (focus on the colored portion) for the layer architecture. This class will contain the following submodules:

- A `SpeechEmbedding` layer to process input speech features and apply time reduction (see `hw3lib/model/speech_embedding`
- Positional encodings (RoPE or sinusoidal) for the encoder and decoder.
- A user-specified number of `SelfAttentionEncoderLayer`s and `CrossAttentionDecoderLayer`s.
- An `nn.Embedding` layer for the target tokens, and a final `nn.Linear` layer to obtain logits over the vocabulary.
-
- An `nn.LayerNorm` after both the encoder and decoder stacks (these are always `nn.LayerNorm`; `norm_type` only applies inside the layers), and an `nn.Dropout` layer.
- A **CTC Head**: a linear layer followed by a log-softmax that projects the encoder output into the vocabulary space, for auxiliary training with the CTC loss.

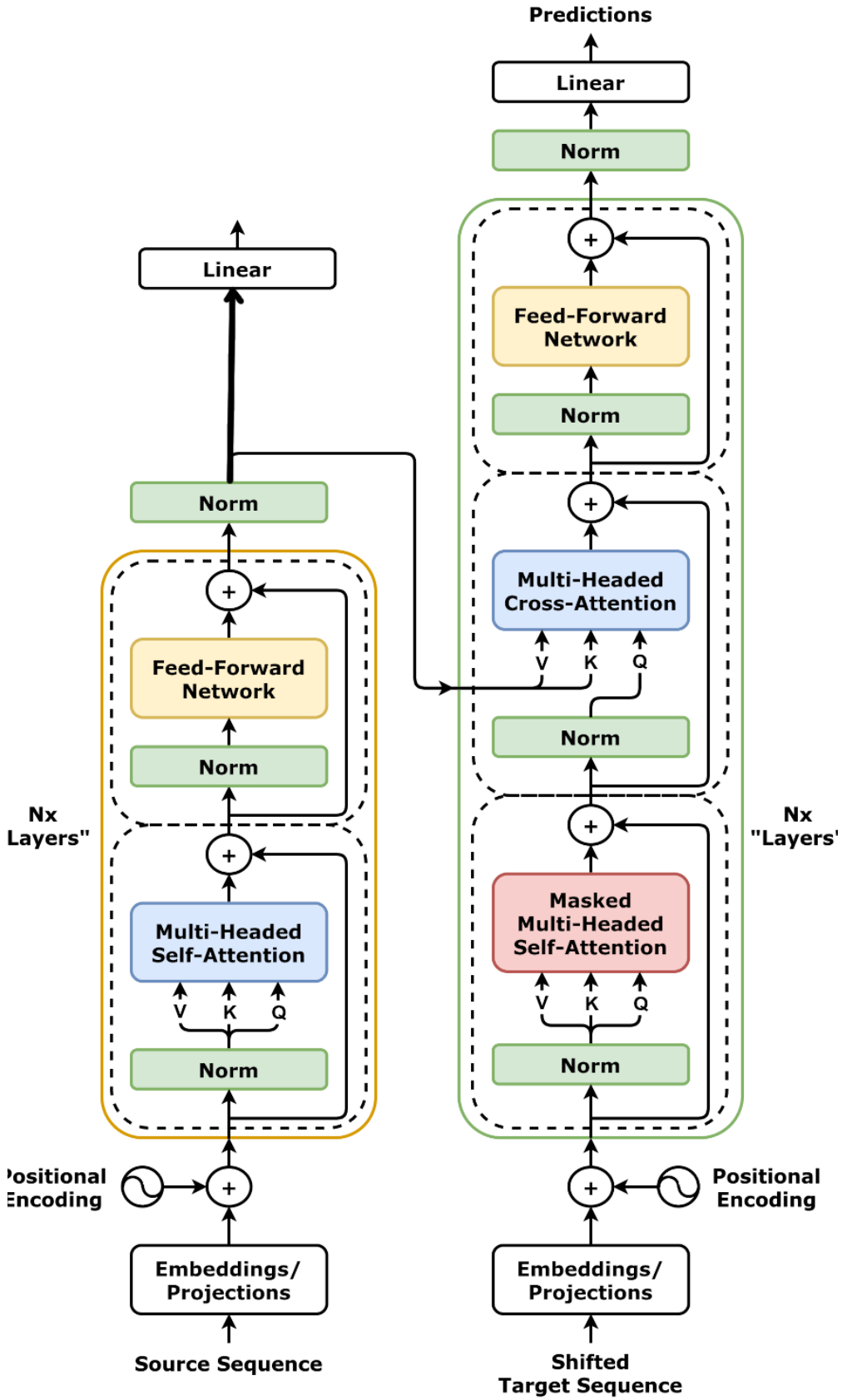


Figure 4: Full Transformer. With RoPE (the default), position is applied inside the self-attention layers instead of being added to the embeddings as shown.

Architecture Options: You can experiment with `encoder_pos_encoding` / `decoder_pos_encoding` (`rope`, `sinusoidal` or `none`), `norm_type` (`rmsnorm` or `layernorm`), `ffn_type` (`swiglu` or `gelu`) and `qk_norm`.

Ensure that you follow the docstrings in the source file carefully.

4.4 Optional/Recommended: Decoding Implementation

Implement the decoding methods of the `SequenceGenerator` class in `hw3lib/decoding/sequence_generator.py`, and test them using the `Decoding Implementation` section of the notebook.

Greedy Search (required): Implement `generate_greedy`. At each step, pick the most likely next token, add its log-probability to the score, and append it to the sequence. Stop when every sequence has produced `[EOS]` or `max_length` is reached.

Optional/Recommended: Beam Search: Implement `generate_beam`, which keeps the `beam_width` most likely sequences at each step instead of only one. The following pseudocode provides a guide:

Pseudocode 1: Beam Search Decoding

```
function generate_beam(x, beam_width, temperature, repeat_penalty):

    # Initialize scores and flags
    initialize scores as zeros of shape (batch_size, beam_width)
    initialize finished flags as False for each sequence (batch_size, beam_width)

    # Compute initial logits and probabilities
    logits = score_fn(x) # (batch_size, vocab_size)

    # Process logits
    logits = apply_repeat_penalty(logits, x, repeat_penalty)
    logits = logits / temperature
    log_probs = log_softmax applied to logits

    # Select top beam_width tokens
    # (batch_size, vocab_size) -> (batch_size, beam_width)
    next_tokens = top beam_width tokens from log_probs
    scores = scores corresponding to the top beam_width tokens

    # (batch_size, seq_len) -> (batch_size, beam_width, seq_len)
    expand x along beam dimension

    # (*, seq_len) -> (*, seq_len+1)
    append next_tokens to x

    update finished flags where EOS token is encountered

    for t in range(1, max_length):
        if all sequences are finished:
            break

        # Compute logits for next tokens
        next_token_scores = []
        for each beam:
            compute logits and append to next_token_scores

        # (batch_size, beam_width, vocab_size)
        next_token_scores = stack next_token_scores along beam dimension

        # Process logits
        next_token_scores = apply_repeat_penalty(next_token_scores, x, repeat_penalty)
        next_token_scores = next_token_scores / temperature
```

```
        next_token_scores = log_softmax applied to next_token_scores

        # Compute cumulative scores
        cum_scores = scores + next_token_scores

        # (batch_size, beam_width * vocab_size)
        flatten cum_scores for beam selection

        # Select top beam_width candidates
        # (batch_size, beam_width)
        indices = indices with top beam_width scores from cum_scores
        scores = scores corresponding to selected indices

        # Re-map to get beam indices and token ids
        # (batch_size, beam_width)
        beam_indices = indices // vocab_size
        next_tokens = indices % vocab_size

        update finished flags where EOS token is encountered

        reorder x based on beam_indices
        # (batch_size, beam_width, seq_len) -> (batch_size, beam_width, seq_len+1)
        append next_tokens to x

    sort sequences in the beam hypotheses in descending order of scores
    return x, scores
```

In this pseudocode:

- `x`: Input tensor of shape $(\text{batch_size}, \text{seq_len})$ representing token sequences of the same length.
- `beam_width`: Number of beams to keep at each decoding step.
- `temperature`: Scalar value used to scale logits before selecting the next token.
- `repeat_penalty`: Scalar factor applied to penalize repeated tokens during decoding.
- `score_fn`: Function that takes the current sequences `x` of shape $(\text{batch_size}, \text{seq_len})$ and returns logits of shape $(\text{batch_size}, \text{vocab_size})$.
- `apply_repeat_penalty`: Function that modifies logits to penalize repeated tokens; input and output shapes are $(\text{batch_size}, \text{vocab_size})$.

NOTE: Implementing this function efficiently is crucial since beam search requires multiple sequence evaluations per step. We recommend using vectorized operations instead of `for` loops. Useful PyTorch functions include `torch.topk`, `torch.gather`, `torch.cat`, `torch.sort`, `torch.log_softmax`, and logical tensor operations.

4.5 Training, Evaluation & Generation

Fantastic work! You've reached the final step—training your encoder-decoder transformer for autoregressive language modeling. Just one last task remains!

4.5.1 ASRTrainer

(`hw3lib/trainers/asr_trainer.py`)

The `ASRTrainer` class manages the training, validation and recognition loops of the ASR model. Your task is to complete the parts marked with `TODO` comments, without changing the rest of the code.

WARNING: The provided classes already handle training, saving models and predictions, and generating plots, so you should not need to change the rest of the notebook. If you choose to diverge from our code, make sure you understand the impact of your changes.

Implementation Tasks:

- **Initialize the losses in `__init__`:** the cross-entropy loss, and the CTC loss if `ctc_weight` > 0.
- **Complete the training loop in `_train_epoch`**, where the total loss is CE + `ctc_weight` $\times$ CTC.
- **Implement the validation loop in `_validate_epoch`.**
- **Finalize the full training pipeline in `train`.**
- **Integrate your decoding in `recognize`:** use greedy search, or beam search if you implemented it.

Important: Optimize your model for **Character Error Rate (CER)**, measured on the validation set.

5 Submission

Once you have completed all implementations, trained your model, and achieved a satisfactory **Character Error Rate (CER)** on the validation set, you are ready to submit your results on Kaggle. The notebook generates a `results.csv` file with the following structure:

id	transcription
0	HELLO WORLD
1	THIS IS AN EXAMPLE
2	ASR MODELS TRANSCRIBE SPEECH
3	OPTIMIZATION REDUCES CER

Table 1: Example format of `results.csv`.

Submission Steps:

- Run the provided cell in the **Submit to Kaggle** section of the notebook. It generates `results.csv` and submits it to the competition. If the automatic submission fails, follow the manual submission steps in the notebook.
- After the Kaggle deadline, submit your code to **Gradescope** by following the instructions at the end of the notebook.

Checkpoint: To meet the checkpoint requirement, you must complete the HW3 Quiz with a full score on the MCQs by the checkpoint deadline. Otherwise, there will be a 3% deduction from your final score.

The **HIGH**, **MEDIUM**, **LOW** and **VERY LOW** cutoff thresholds will be announced on **Piazza** after the checkpoint deadline has passed.

6 Tips & Tricks

Implementing and training a Transformer model for Automatic Speech Recognition (ASR) is both challenging and rewarding. Below are strategies to enhance your model's performance and efficiency.

6.1 Managing Trade-offs through Feature Representation

Balancing training efficiency and model performance is crucial. Training a Transformer can be resource intensive, but careful choices in feature selection, tokenization, and embedding can mitigate this.

6.1.1 Impact of Tokenization

The choice of tokenization strategy affects sequence length, vocabulary size, and how well the text aligns with the speech features. `H3Tokenizer` supports character-level tokenization and BPE subword tokenization with vocabulary sizes of 1k, 5k and 10k.

- **Character-Level Tokenization:** Represents text as individual characters. This leads to longer sequences, but a small vocabulary and no out-of-vocabulary (OOV) issues.
- **Subword Tokenization (BPE):** Breaks words into frequently occurring sub-units. In general, **larger vocabularies lead to shorter sequences**, since fewer tokens are needed per transcription. However, they also increase the size of the embedding and output layers, which count toward the 30M parameter limit.

6.1.2 Speech Feature Representation

You are provided with 80-dimensional filter bank features extracted from the raw audio. However, experimenting with the number of features used can influence performance. Using fewer features can reduce memory and computation costs. However, dropping too much information might result in loss of critical information needed for accurate transcription. You can change the number of features used with `num_feats` in the config.

6.1.3 Sequence Downsampling for Efficient Training

The sequence of filter banks is processed by the `SpeechEmbedding` layer, which consists of stacked CNNs and/or LSTMs. Raw speech feature sequences can be long, making training slow and memory-intensive. Downsampling reduces the sequence length while preserving relevant information, and is controlled by `time_reduction` and `reduction_method` (`lstm`, `conv` or `both`) in the config:

- **CNN-based Downsampling:** Convolutional layers with strides greater than 1 reduce the sequence length early in the model.
- **Pooling Layers:** Time-axis pooling, such as max or average pooling, can further compress the sequence.
- **Subsampling in LSTMs:** Skipping or merging frames between LSTM layers can also reduce computation.

However, too much downsampling can lose critical information and hurt performance.

6.2 Training Strategy 3: Progressive Trainer

Progressive Training is an advanced but effective strategy that gradually increases the model's capacity during training. This approach offers several key advantages, particularly for large, complex models like Transformers. The core idea behind progressive training is that a model trained on a simpler version of the task can generalize more effectively to more complex tasks, thus resembling a form of transfer learning. By progressively increasing the difficulty, this strategy helps reduce the risk of overfitting early in the training process.

One of the primary benefits of progressive training is its ability to speed up the initial stages of training by using fewer parameters. This enables faster convergence, allowing the model to learn the basic structure of the problem before dealing with more complex aspects. Additionally, progressive training stabilizes the learning process by gradually introducing more complexity, which minimizes the risk of unstable gradients. This also leads to better resource utilization, as only a small portion of the model or data is trained initially, saving both memory and computational resources and facilitating quicker experimentation.

It is essential to define the concept of "difficulty" when using this strategy. In the context of training an ASR model, difficulty can be gradually increased by scaling up the dataset, increasing model complexity, or adjusting regularization techniques such as augmentation, dropout, or label smoothing. Another way to define difficulty could be heuristic-based—for example, shorter sentences may be easier to transcribe than longer ones. In this case, you could split your dataset by sentence length and initially train the model on shorter sequences before gradually introducing longer ones.

Additionally, it is important to determine the schedule for increasing difficulty. Simple strategies may use a fixed schedule, such as progressively doubling the number of encoder-decoder layers every 5 epochs while simultaneously increasing regularization. Alternatively, more sophisticated approaches may incorporate self-paced learning, where the difficulty level increases proportionally to the model's performance on the current training data. This adaptive method can provide more flexibility and allow the model to progress at its own pace, ensuring it is always challenged without being overwhelmed.

We recommend the following schedule: You can start with a smaller model, initially using just 1 encoder and 1 decoder layer. The idea is to progressively increase the model's capacity by adding more layers after each pretraining iteration. Here's the step-by-step process:

1. **Start Small:** Begin training with a minimal architecture, typically with only 1 encoder layer and 1 decoder layer.
2. **Progressive Layer Addition:** After each pretraining iteration, increase the number of layers in the encoder and decoder, making the model progressively deeper. This allows the model to gradually learn complex features without overwhelming it early on.
3. **Freezing Layers:** Optionally, you can freeze the previously added layers to prevent them from being updated in the early stages of training. This can help the new layers to better learn the task-specific features before fine-tuning the entire model.
4. **Regularization:** Initially, keep regularization techniques such as `dropout` and `label smoothing` low or disabled. As the model complexity increases, you can gradually enable and tune these techniques to prevent overfitting.
5. **Unfreeze All Layers:** Once the model has trained sufficiently with the progressively added layers, unfreeze all layers and train the full model. At this point, all layers should be updated during training, with appropriate regularization applied.

This approach allows you to start with a simpler model, gradually making it more complex. The progressive addition of layers helps the model learn effectively without requiring all parameters to be learned at once, which can lead to faster convergence and potentially better generalization.

The schedule above is supported by the `ProgressiveTrainer` class provided in the handout. The `Progressive Trainer` class inherits from the `ASRTrainer` class and implements the curriculum learning approach described above where model complexity and regularization are gradually increased through defined training stages. However, the implementation is not directly available to you in the notebook. You will need to read through and understand the framework, then write the necessary code to implement this yourself. To help you with this, here are a couple of useful resources:

1. The `ProgressiveTrainer` class can be used to define your schedule and train via in a one-shot manner. You can also use it to train the full model.
2. The `create_optimizer` function in `hw3lib/utils/create_optimizer`. This function is useful for setting layer-specific learning rates, which is particularly helpful when applying different learning rates to the frozen and trainable layers.

NOTE: This strategy can be combined with the **Pretrained Decoder Initialized strategy (discussed previously)**!

6.3 Gradient Accumulation

Gradient Accumulation is a technique that allows you to train models with large batch sizes even when limited by memory constraints. This method is particularly useful when training large models like Transformers, which can require substantial memory, especially with long sequences and large batch sizes. Instead of updating the

model weights after each mini-batch, gradient accumulation accumulates gradients over multiple mini-batches before performing a weight update. This effectively simulates the effect of a larger batch size without requiring the memory capacity to hold all the gradients at once. This is already implemented in the framework that we have provided. You will be able to use this in your pipeline by setting `gradient_accumulation_steps` in your config.

6.4 Debugging NaN Loss

Encountering NaN (Not a Number) loss during training can be a frustrating issue. Here are some steps to debug and fix this problem:

1. **Check Learning Rate:** A high learning rate can cause the model to diverge, leading to NaN values. Try reducing the learning rate and observe if the issue persists.
2. **Inspect Weight Initialization:** Improper weight initialization can lead to NaN loss. Ensure that the weights are initialized using a method appropriate for the activation functions in your network (e.g., Xavier initialization for sigmoid activations).
3. **Monitor Gradients:** Check if the gradients are exploding, which can lead to NaN values. You can print the gradients or log them to Weights & Biases. If you notice large gradients, consider implementing gradient clipping to prevent them from exceeding a certain threshold.
4. **Use a Different Optimizer:** Sometimes, switching to a different optimizer can help stabilize training. For example, if you're using SGD, try using Adam or RMSprop and see if the issue is resolved.
5. **Add Small Constants:** In operations that involve division or logarithms, adding a small constant (e.g., 1e-8) to the denominator or argument can prevent division by zero or taking the logarithm of zero, which can lead to NaN values.
6. **Check for Inf Values:** Ensure that there are no infinite values in your data or intermediate computations. Infinite values can propagate through the network and result in NaN loss.
7. **Reduce Model Complexity:** A very complex model might be more prone to numerical instability. Try simplifying your model architecture and see if the issue persists.
8. **Check for float16 Overflow:** Training uses mixed precision, and float16 values overflow above about 65,000. Make sure numerically sensitive steps, such as the mean of squares in `RMSNorm`, are computed in float32.

By systematically addressing each of these potential causes, you can identify and fix the source of NaN loss in your training process.

6.5 Out Of Memory (OOM)

Out of Memory errors occur when your model requires more memory than is available on your GPU. To resolve this, you can reduce the batch size, use gradient accumulation, reduce the dimensionality of the speech features, down-sample the length of the speech embedding sequence, switch to a different tokenization strategy, or simplify your model architecture.

6.6 Monitoring Training

6.6.1 Visualizing & Interpreting Attention Plots

Visualizing attention plots, especially decoder masked self-attention and decoder cross-attention, is a valuable tool to monitor and interpret the training process of an encoder-decoder transformer model in Automatic Speech Recognition (ASR). These attention maps allow us to understand how the model attends to different parts of the input sequence and previous tokens during the generation of each output token. If the attention mechanism is poorly aligned, it may indicate problems in the model's learning process, such as insufficient training or issues with the architecture. The framework provided will dump these plots periodically during training. We highly recommend monitoring this periodically!

Decoder Masked Self-Attention The decoder masked self-attention is primarily responsible for generating the output sequence based on previously generated tokens. In the case of ASR, the self-attention mechanism is autoregressive, meaning it can only attend to the prior generated tokens (and not future ones). As a result, attention

plots for this mechanism typically show a diagonal pattern, where each token primarily attends to itself and its preceding tokens.

Expected Shape and Trends:

- **Diagonal Dominance:** The attention plots will often show a diagonal pattern, which indicates that, at each decoding step, the model mainly focuses on previously generated tokens.
- **Shifted Attention:** As the model progresses through training, attention may gradually shift to earlier tokens, reflecting the model's growing ability to maintain context over longer sequences.
- **Noise Reduction:** At the early stages of training, attention may appear more scattered. However, as the model trains, attention tends to become more focused, indicating improved understanding of prior context.

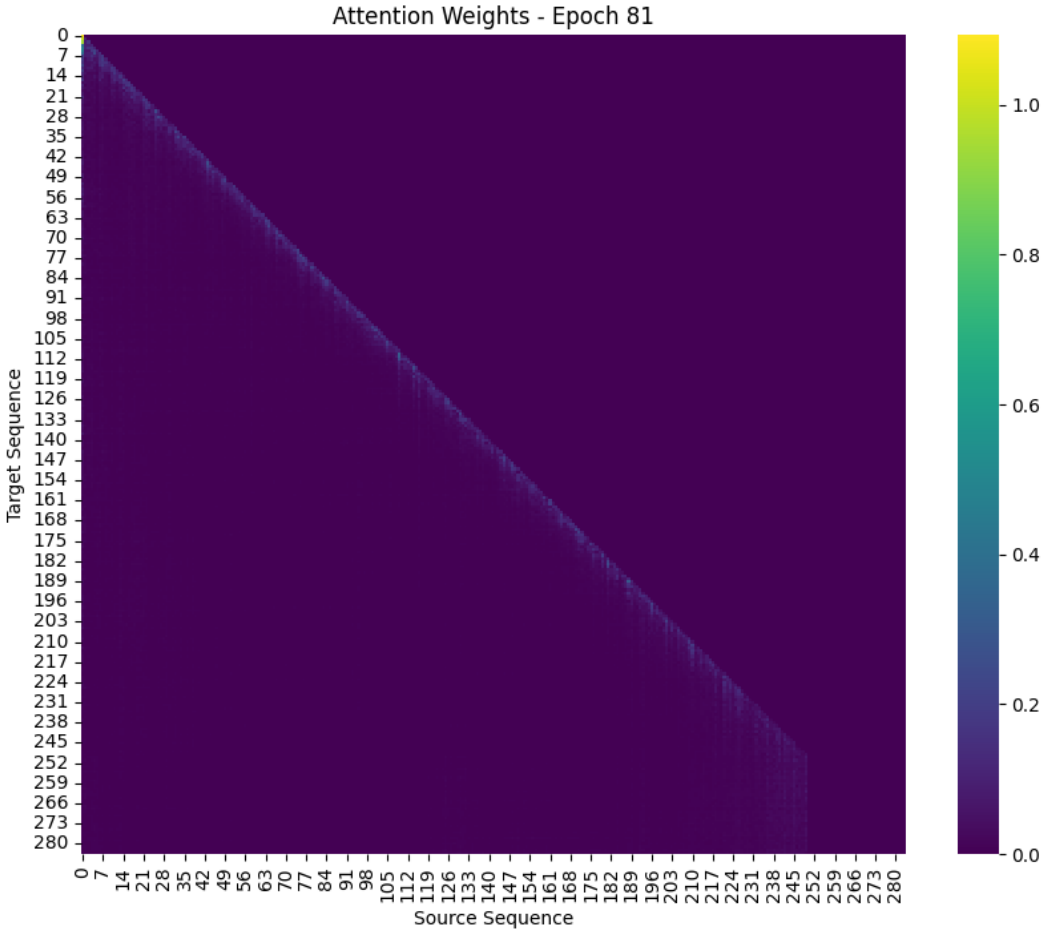


Figure 5: Example of a well-aligned Decoder masked self-attention heatmap.

Decoder Cross-Attention The decoder cross-attention mechanism is crucial in ASR tasks, as it allows the decoder to pay attention to the encoded speech features during the generation of each output token. In contrast to self-attention, which focuses on previously generated tokens, cross-attention allows the model to attend to relevant portions of the input speech sequence.

Expected Shape and Trends:

- **Initial Noisiness:** Initially, attention in the cross-attention plots may appear noisy and all over the place as the model learns which input sequences it should focus on for the corresponding output sequence.

- **Alignment with Speech Segments:** In the cross-attention layer, attention should progressively align with specific segments of the audio features that correspond to the current token being generated in the transcription. Given that in ASR, the input (speech) and output (text) are order-synchronous, you should expect the attention maps to exhibit a diagonal-like structure. This structure will become increasingly pronounced as the model trains, reflecting a growing ability to attend to the correct segments of the input audio at each decoding step.

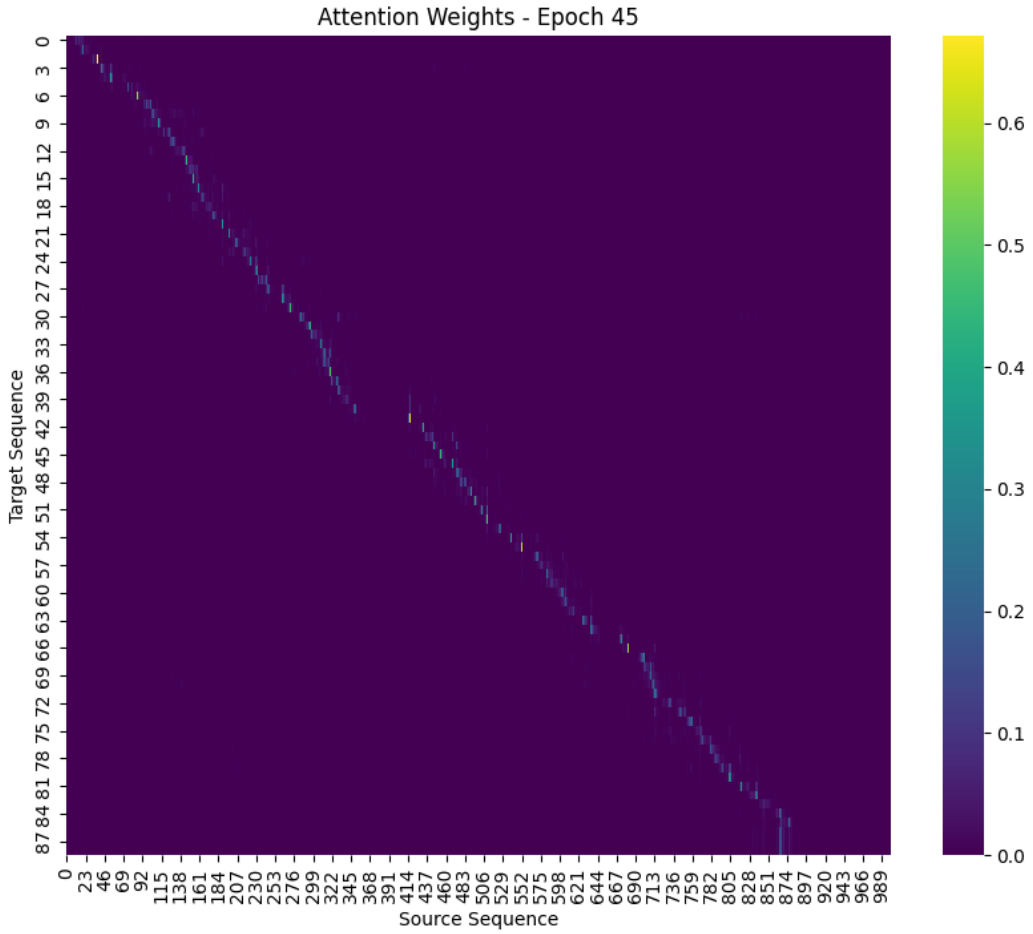


Figure 6: Example of a well-aligned Decoder cross-attention heatmap.

6.6.2 Generated Text Comparison

Along with the attention plots, we also recommend periodically inspecting the generated text and how it compares with the ground truth during training. Early on, the generated text may be noisy or contain errors, but as training progresses, you should expect the text to become more accurate and coherent. Tracking improvements in the generated output, in combination with attention plot analysis, can provide a deeper understanding of the model's performance and highlight areas that may need further optimization.

6.7 Miscellaneous

Training a Transformer model to reach optimal performance can be time-consuming, likely taking longer than previous tasks you have worked on. We recommend saving and tracking checkpoints regularly during training to avoid losing progress in case of interruptions. This also allows you to resume training from the last saved state. The

framework provided to you implements rigorous checkpointing both locally **and in Weights & Biases (W&B)**, but it is your responsibility to track and manage these checkpoints effectively.

When debugging common issues, carefully examine the loss curves, check the gradients for abnormalities, and ensure that the data pipeline is working correctly. If the model is not converging, it might be tempting to add more complexity. However, do not overlook the basics; consider adjusting the learning rate, experimenting with different optimization algorithms, or training without regularization to assess the model's capacity before introducing more complexity.

This homework is a true litmus test for your understanding of concepts in Deep Learning. It is not easy to implement and the competition will be tough. But we guarantee you that if you manage to complete this homework by yourself, you can confidently claim that you are now a **Deep Learning Specialist**! Good luck and enjoy the challenge!